程序申请 1G 内存为什么没崩：overcommit 与 OOM

嵌入式 Linux 手记　2026-11-03　公众号文章 · 原链接 mp.weixin.qq.com

先猜结果。机器 8GB 内存，这段代码在 Linux 上跑：

```c
#include <stdlib.h>

int main(void) {
    char *p = malloc(1 << 30);    /* 申请 1GB */
    if (p == NULL)
        return 1;                 /* 会走到这里吗？ */
    p[0] = 1;                     /* 只碰 1 字节 */
    return 0;
}
```

`malloc(1GB)` 成功返回了吗？绝大多数 Linux 上：成功，程序正常退出，机器毫发无伤。追问一句：那 1GB 真的存在吗？不存在。物理内存一分都没为它准备，内核只是把这笔账记在了"承诺簿"上。再把剧本改一下：程序真的把 1GB 全部写满，会发生什么？崩的不一定是它：内核会挑一个进程直接击杀，可能就是这个程序，也可能是旁边一个无辜的路人。这一篇拆开过度承诺（overcommit）的账本和最后一刀（OOM killer）的评分表。

一、过度承诺：内核为什么敢答应

答案的一半在篇 16 已经铺好：demand paging。`malloc(1GB)` 时，glibc 向内核要一段地址空间（大块走 `mmap`），内核做的只是登记一条虚拟地址区间（VMA），页表里一个映射都没建："这 1GB 将来归你"，仅此而已。程序 `p[0] = 1` 只碰 1 字节，缺页异常触发，内核才补一个填零的物理页。所以 8GB 的机器答应 1GB 毫无压力：答应的是地址空间的承诺，不是物理内存的交割。

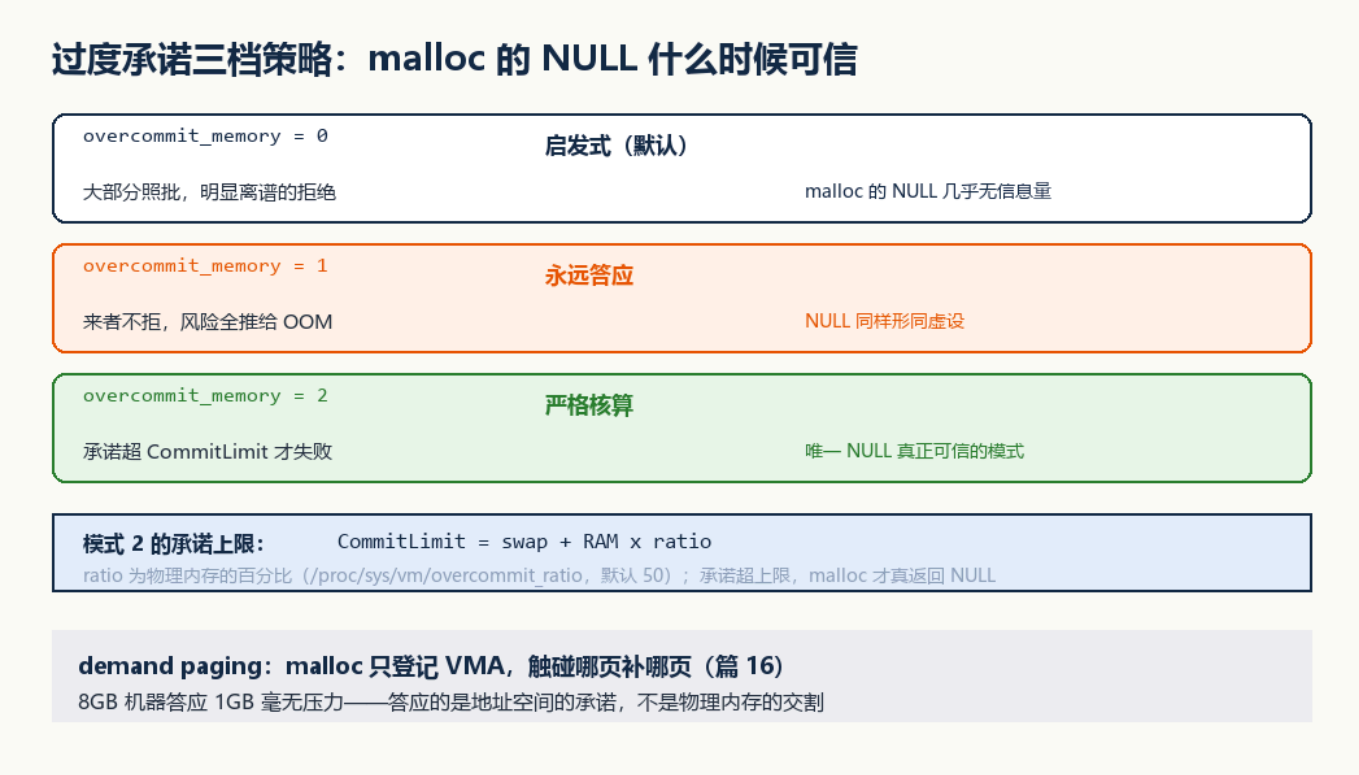
这套策略叫 overcommit（过度承诺）。它赌的是概率：程序申请的内存通常不会全部真用，按申请总量预留物理内存就太浪费。数据库说它要 64GB 缓冲池，实际热数据只占 10GB，若严格预留，54GB 白白躺着。

二、三种模式：承诺管到什么程度

内核对"承诺敢给多少"提供了三档策略，由 `/proc/sys/vm/overcommit_memory` 控制：

模式	含义	行为
0（默认）	启发式	大部分请求照批，明显离谱的（超物理+swap 太多）拒绝
1	永远答应	所有请求来者不拒，风险全推给 OOM
2	严格核算	承诺总量受硬上限约束，超了 malloc 真返回 NULL

模式 2 是唯一"malloc 返回值可信"的模式：内核维护一本承诺账本（commit），上限为 `CommitLimit = swap + 物理内存 × 比率`，比率默认 50%（`/proc/sys/vm/overcommit_ratio` 可调）。承诺账本超出上限，分配才真正失败。代价是内存利用率掉下来：为"可能用满"买单，正是篇 21 那套确定性思维在内核的投影。



图：三档策略对照。只有模式 2 下 malloc 的 NULL 才真正可信；默认模式 0 的承诺几乎来者不拒。

模式 0 之下，`malloc` 返回非 NULL 几乎不携带任何信息量：承诺簿记了一笔，仅此而已。篇 16 说过"malloc 成功不等于内存到手"，至此机制完整闭环。

三、OOM killer：最后一刀的评分表

过度承诺总有穿帮的时刻：物理内存和 swap 都见了底，而某个程序真的要把承诺兑现。此时内核没有新页可补，缺页异常无法修复，桌面惯例是启动 OOM killer（Out-Of-Memory Killer），从全体进程里挑一个杀掉，腾出内存续命。

挑选逻辑是打分制：核心依据是谁占的内存多：把各进程占用的物理页与 swap 归一化成 `oom_score`，分越高越优先被杀；管理员可以用 `oom_score_adj`（-1000 到 1000）干预，比如给关键守护进程调到 -1000 豁免。被选中者收到 `SIGKILL`，不可捕获、不可忽略、立即终止，dmesg 里会留下完整案发记录（谁杀的、为什么、当时各进程得分）。

这里藏着一个工程上反直觉的真相：OOM 杀的未必是闯祸的。一个只申请不兑现的进程毫发无伤，老实占着大块物理内存干活的路人进程反而因为分高被击杀。多进程服务器上 OOM 事件像抽签，这是"高承诺低兑现"模式的固有代价。



图：OOM killer 的触发链与评分表。占内存越多越先被杀；闯祸者与路人，看的是占用不是过错。

四、malloc 返回值还要不要查

三段话把工程姿势定清楚。第一，模式 0 的 Linux 上，判 `malloc` 返回值防不了"承诺兑现失败"，但仍是必要习惯：它防的是模式 2 下的真失败、显式 `mmap` 的失败，以及移植到无 overcommit 平台（多数 RTOS，篇 21 的世界）时的真实拒绝。第二，关键路径要确定性，就别

把命运交给默认策略：改用 `mmap` 显式控制（`MAP_NORESERVE` 反向选择要不要参与承诺），或把系统切到模式 2 让承诺账本可审计。第三，大缓冲池的进程把自己 `oom_score_adj` 调负值，避免被路人抽签抽中。

还有一个嵌入式专属的放大器：swap。桌面系统挂一块 swap 分区，等于给过度承诺垫了一层缓冲——兑现超出物理内存的部分先落到磁盘，OOM 的引线被拉长；而嵌入式 Linux 产品普遍禁用 swap（eMMC/SD 卡写入寿命扛不住频繁换页），安全网直接不存在，承诺兑现失败一秒都不缓冲。同一段代码，服务器上是"偶发抽签"，无 swap 的工控板上就是"必炸且炸别人"，部署前值得先看一眼 `/proc/swaps`。至于排查，案发后一条命令定位：`dmesg | grep -i oom`，内核会打印 killer 的完整判词——选中谁、得分多少、当时各进程的内存账单。

写在最后

1. `malloc(1GB)` 成功只是内核在承诺簿记了一笔：demand paging 下触碰哪页才分配哪页（篇 16 的机制），物理内存一分未动。
2. overcommit 三模式：0 启发式（默认）、1 永远答应、2 严格核算（`CommitLimit = swap + 内存 × 比率`，默认 50%）；只有模式 2 下 malloc 的 NULL 才真正可信。
3. 兑现失败时 OOM killer 出场：按占用内存打分选牺牲者，`oom_score_adj` 可豁免；被杀者收 `SIGKILL`，不可捕获，案发记录在 dmesg。
4. OOM 杀的未必是闯祸的：高承诺低兑现的进程安然无恙，老实占内存的路人反而分高，多进程系统里的抽签风险。
5. malloc 判空仍是必要习惯（防模式 2、RTOS、显式 mmap 的真失败），关键路径的确定性用显式 mmap 或模式 2 兑换。

内存的"承诺"还有最后一处魔法：全局变量不写初始化代码，凭什么一开机就是 0？下一篇《全局变量为什么自动是 0：.data 与 .bss 的分工》，看两个段怎么分工、.bss 怎么做到不占文件空间，以及"清零"背后的安全动机。

合集持续更新，往期与后续篇目都在目录里：C 语言底层实战 · 合集目录

后台回复「交流」，我很乐意分享我的学习计划和一线工作经验，直接来聊。